

Apache Spark Lab Assignment Report

Course: Big Data Analytics and Applications
Lab: Laboratory Assignment 1 — Setting Up Apache Spark and Performing Data Analysis
Student Name: Ibrahim Mourid
Student ID: 20243120060
Instructor: Tang Li
Submission Date: 2026/11/-3

Table of Contents

1. Introduction and Learning Objectives
2. Environment Setup
3. Task 1: Install and Verify Spark
4. Task 2: Analyze Order Data with DataFrames
5. Task 3: Query the Data with Spark SQL
6. Task 4: Reflection Questions and Lab Summary
7. Conclusion
8. References and AI-Use Acknowledgement

---

1. Introduction and Learning Objectives

1.1 Introduction

Apache Spark is a distributed data-processing engine that supports data analysis using APIs such as RDDs, DataFrames, and Spark SQL. PySpark is the Python interface for working with Spark. In this lab, I used

PySpark to create a Spark session, load a simulated online-retail dataset, perform data analysis with the DataFrame API, and run SQL queries against the same data.

The dataset represents 1,000 orders. Each order contains an order identifier, user identifier, product category, unit price, quantity, and city. The analysis calculates order amounts, summarizes order counts and revenue, identifies high-value orders, and finds users with the highest total spending.

The lab runs Spark in local mode. This makes it possible to learn Spark concepts on one computer without deploying a separate Spark cluster.

1.2 Learning objectives

The objectives of this lab were to:

- Install and verify Java, Python, and PySpark.
- Create and use a SparkSession.
- Read a CSV file into a Spark DataFrame and inspect its schema and records.
- Check the dataset for missing values.
- Use the DataFrame API to filter, group, aggregate, and sort records.
- Write SQL queries using a temporary Spark SQL view.
- Compare the DataFrame API with Spark SQL.
- Read a physical execution plan and recognize operations such as scanning, aggregation, data exchange, and sorting.
- Understand transformations, actions, and Spark's execution process.

---

2. Environment Setup

2.1 Software and execution environment

The lab instructions recommend Windows 10/11, JDK 17, Python 3.11, and PySpark 4.2.0, or an instructor-approved compatible version. My work was performed on the following environment:

Component	Version or details
Operating system	Windows (version 10.0.26100.9711)
Java / JDK	OpenJDK 17.0.20.1 LTS (Microsoft build 17.0.20.1+1-LTS)
Python	Python 3.15.1
PySpark	PySpark 4.2.0
Code editor	Visual Studio Code
Spark execution mode	Local mode using .master("local[*]")

I used a Python virtual environment named spark-env to keep the lab's Python dependencies separate from other Python projects. I activated the environment before running the scripts.

2.2 Installation verification

I verified the installed tools with the following commands:

```
java -version
python --version
python -m pip show pyspark
```

These commands display the Java version, Python version, and installed PySpark package information. The screenshots below provide evidence of the environment used for the lab.

```
Microsoft Windows [Version 10.0.26100.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\mouri>java -version
openjdk version "17.0.20.1" 2026-08-18 LTS
OpenJDK Runtime Environment Microsoft-14940689 (build 17.0.20.1+1-LTS)
OpenJDK 64-Bit Server VM Microsoft-14940689 (build 17.0.20.1+1-LTS, mixed mode, sharing)

C:\Users\mouri>
```

Figure 1. Verification of the installed Java version.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights rese

PS C:\Users\mouri> python --version
Python 3.15.1
PS C:\Users\mouri>
```

Figure 2. Verification of the installed Python version.

```
Successfully built pyspark
Installing collected packages: py4j, pyspark
Successfully installed py4j-0.10.9.9 pyspark-4.2.0

(spark-env) C:\Users\mouri>python -m pip show pyspark
Name: pyspark
Version: 4.2.0
Summary: Apache Spark Python API
```

Figure 3. Verification of the installed PySpark version.

2.3 Project structure

The project was organized so that Python scripts, generated data, screenshots, and the report are stored separately.

```
20243120060_Ibrahim_Mourid_SparkLab/
├── Lab_Report.pdf
├── code/
│   ├── test_spark.py
│   ├── generate_data.py
│   ├── dataframe_analysis.py
│   └── spark_sql_analysis.py
├── data/
│   └── orders.csv
└── screenshots/
```

The CSV file is stored in the data directory, while the scripts are stored in code. The analysis scripts use a path based on the script's location so that they can locate data/orders.csv when run from the project root.

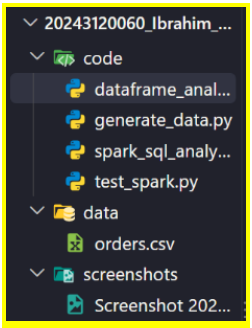


Figure 4. Project directory structure.

3. Task 1: Install and Verify Spark

3.1 Creating a SparkSession

The first program, test_spark.py, creates a Spark session, prints the Spark version, creates a small DataFrame, displays its records, counts the records, and stops Spark.

The key code is:

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("FirstSparkExperiment")
    .master("local[*]")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")
print("Spark Version:", spark.version)

data = [("Alice", 85), ("Bob", 92), ("Cindy", 78)]
df = spark.createDataFrame(data, ["name", "score"])
df.show()
print("Total Records:", df.count())

spark.stop()
```

SparkSession is the main entry point for using Spark SQL and DataFrames. The appName() method sets the application's name. The setting .master("local[*]") tells Spark to run locally and use the available logical CPU cores for parallel processing. getOrCreate() obtains an existing session or creates one if necessary.

The createDataFrame() method creates a DataFrame from the supplied records and column names. show() displays the data, while count() is an action that calculates the number of rows. The expected count for this test dataset is three.

3.2 Execution results

```
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/24 16:20:14 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using built
in-java classes where applicable
Spark Version: 4.2.0
+-----+-----+
| name|score|
+-----+-----+
|Alice|   85|
|  Bob|   92|
|Cindy|   78|
+-----+-----+

Total Records: 3
```

Figure 5. Successful execution of the first Spark experiment.

The program successfully demonstrated that PySpark could start a Spark session and process a simple DataFrame. The output contained the three sample records, and the record count was 3, which matched the number of input records.

---

4. Task 2: Analyze Order Data with DataFrames

4.1 Generating the dataset

The file generate_data.py creates a simulated retail dataset containing 1,000 orders. It uses Python's csv and random modules. A fixed random seed (random.seed(2026)) makes the generated dataset reproducible when the same code and generation procedure are used.

The CSV has the following columns:

Column	Meaning
order_id	Unique identifier for an order
user_id	Identifier of the user associated with the order
category	Product category
price	Unit price
quantity	Quantity purchased
city	City associated with the order

The generator chooses categories from Electronics, Books, Clothing, Food, and Sports, and cities from Beijing, Shanghai, Guangzhou, Shenzhen, and Chengdu. Prices and quantities are generated within the ranges specified in the teacher's starter code.

I saved the generated file as data/orders.csv. I checked that the file existed before running the analysis scripts.

4.2 Loading and inspecting the dataset

The analysis script reads the CSV into a Spark DataFrame with the header enabled and schema inference enabled:

```
df = spark.read.csv(
    str(csv_path),
    header=True,
    inferSchema=True
)

df.show(10)
df.printSchema()
print("Total orders:", df.count())
```

header=True tells Spark that the first row contains column names. inferSchema=True asks Spark to infer data types from the input. show(10) displays the first ten records, printSchema() displays the column names and data types, and count() returns the number of rows.

```
|order_id|user_id|   category| price|quantity|     city|
+--------+-------+-----------+------+--------+---------+
|       1|     31|   Clothing|507.49|       5|  Beijing|
|       2|     58|     Sports|625.27|       4|  Chengdu|
|       3|    141|       Food|753.52|       5| Shenzhen|
|       4|     62|Electronics|618.06|       1|Guangzhou|
|       5|     26|       Food| 21.37|       4|Guangzhou|
|       6|     54|       Food| 259.0|       3| Shenzhen|
|       7|    191|     Sports|641.34|       3|  Beijing|
|       8|    143|     Sports| 299.2|       4| Shanghai|
|       9|    166|     Sports|315.91|       3|Guangzhou|
|      10|    119|       Food| 99.78|       5|  Chengdu|
+--------+-------+-----------+------+--------+---------+
only showing top 10 rows
root
 |-- order_id: integer (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- category: string (nullable = true)
 |-- price: double (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- city: string (nullable = true)

Total orders: 1000
```

Figure 6. Preview and schema of the generated order dataset.

The output reported 1,000 records. The expected dataset size is 1,000 orders. The schema showed the types Spark inferred for the order identifier, user identifier, category, price, quantity, and city columns.

4.3 Checking missing values

I checked each column for null values by counting the rows in which each column was null:

```
from pyspark.sql.functions import col, sum

df.select([
    sum(col(column).isNull().cast("int")).alias(column)
    for column in df.columns
]).show()
```

For each column, isNull() checks whether a value is null, cast("int") converts the Boolean result to 1 or 0, and sum() adds the null indicators. This produces a null count for every column.

```
Total orders: 1000
Missing values:
+--------+-------+--------+-----+--------+----+
|order_id|user_id|category|price|quantity|city|
+--------+-------+--------+-----+--------+----+
|       0|      0|       0|    0|       0|   0|
+--------+-------+--------+-----+--------+----+
```

Figure 7. Missing-value check for the orders dataset.

The check showed no missing values: the null count was 0 for all six columns (order_id, user_id, category, price, quantity, and city). This check is useful because missing data can affect calculations, grouping, and the interpretation of analysis results.

4.4 Analysis 1 — Calculate the amount for each order

The order amount is defined as the unit price multiplied by the quantity:

```
df = df.withColumn("amount", col("price") * col("quantity"))

df.select(
   "order_id", "category", "price", "quantity", "amount"
).show(10)
```

withColumn() adds the calculated amount column to the DataFrame. The expression uses Spark column operations rather than converting the dataset to pandas.

```
+--------+-----------+------+--------+-------+
|order_id|   category| price|quantity| amount|
+--------+-----------+------+--------+-------+
|       1|   Clothing|507.49|       5|2537.45|
|       2|     Sports|625.27|       4|2501.08|
|       3|       Food|753.52|       5| 3767.6|
|       4|Electronics|618.06|       1| 618.06|
|       5|       Food| 21.37|       4|  85.48|
|       6|       Food| 259.0|       3|  777.0|
|       7|     Sports|641.34|       3|1924.02|
|       8|     Sports| 299.2|       4| 1196.8|
|       9|     Sports|315.91|       3| 947.73|
|      10|       Food| 99.78|       5|  498.9|
+--------+-----------+------+--------+-------+
```

Figure 8. Calculation of order amounts.

The amount field represents the total value of an individual order before any additional discounts, shipping fees, or taxes, because those fields are not included in this dataset.

4.5 Analysis 2 — Count orders by category

I used groupBy("category").count() to calculate the number of orders in each product category:

```
df.groupBy("category").count().show()
```

groupBy() groups rows with the same category. count() calculates the number of rows in each group.

```
Analysis 2: Orders by category
+-----------+-----+
|   category|count|
+-----------+-----+
|       Food|  196|
|     Sports|  191|
|Electronics|  222|
|   Clothing|  213|
|      Books|  178|
+-----------+-----+
```

Figure 9. Number of orders in each product category.

The category with the largest order count was Electronics, with 222 orders. The category with the smallest count was Books, with 178 orders. The other categories were Clothing (213), Food (196), and Sports (191); together the five counts total 1,000 orders.

4.6 Analysis 3 — Calculate revenue by category

I calculated total revenue for each category by summing amount, then sorted the results in descending order:

```
df.groupBy("category") \
  .agg(sum("amount").alias("total_revenue")) \
  .orderBy(col("total_revenue").desc()) \
  .show()
```

agg() applies an aggregation function to each group. Here, sum("amount") adds the order amounts in each category. alias() gives the result a readable name, and orderBy() sorts the output. The desc() method puts the highest revenue first.

```
Analysis 3: Revenue by category
+-----------+------------------+
|   category|     total_revenue|
+-----------+------------------+
|   Clothing|         363458.85|
|Electronics| 315775.4799999999|
|       Food| 285513.6799999998|
|     Sports|281668.09000000014|
|      Books|244056.71000000008|
+-----------+------------------+
```

Figure 10. Total revenue by product category.

According to my results, Clothing generated the highest total revenue of 363,458.85 yuan. This may indicate that the category contributes strongly to sales in this simulated dataset. However, the data is randomly generated, so this should not be treated as evidence of real customer demand.

4.7 Analysis 4 — Order count and revenue by city

I grouped the data by city and calculated both the order count and total revenue:

```
from pyspark.sql.functions import count

df.groupBy("city").agg(
  count("*").alias("order_count"),
  sum("amount").alias("total_revenue")
).show()
```

The count("*") expression counts the orders in each city, while sum("amount") calculates the total order value. This combined summary makes it possible to compare the volume and value of orders across cities.

```
Analysis 4: Orders and revenue by city
+---------+-----------+------------------+
|     city|order_count|     total_revenue|
+---------+-----------+------------------+
|Guangzhou|        211| 305989.2299999999|
|  Beijing|        187|290602.03999999986|
|  Chengdu|        184| 283151.3300000002|
| Shenzhen|        217| 308339.0900000001|
| Shanghai|        201| 302391.1199999998|
+---------+-----------+------------------+
```

Figure 11. Order count and total revenue for each city.

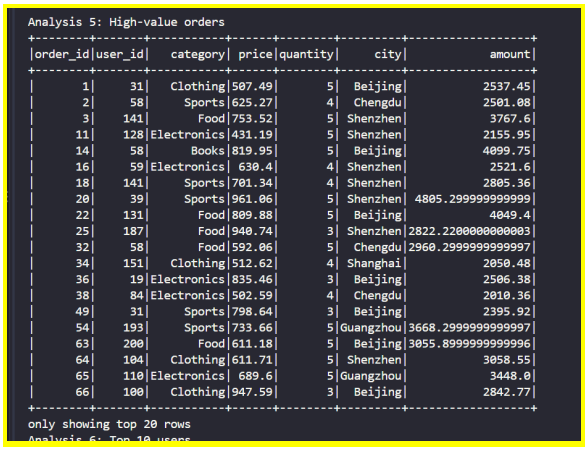
The city with the highest total revenue was Shenzhen, with 308,339.09 yuan. The city with the largest number of orders was also Shenzhen, with 217 orders. In this dataset the same city leads on both measures, although in general the two can differ because a city can have fewer orders but a higher average order value.

4.8 Analysis 5 — Find high-value orders

I filtered the DataFrame to display orders whose amount was greater than 2,000 yuan:

```
df.filter(col("amount") > 2000).show()
```

filter() retains only the rows that satisfy the condition. In this case, the filter uses the calculated total order amount, not the unit price. The condition is strictly greater than 2,000, so an amount equal to 2,000 would not be included.



```
Analysis 5: High-value orders
+--------+-------+-----------+------+--------+---------+-----------------+
|order_id|user_id|   category| price|quantity|     city|           amount|
+--------+-------+-----------+------+--------+---------+-----------------+
|       1|     31|   Clothing|507.49|       5|  Beijing|           2537.45|
|       2|     58|     Sports|625.27|       4|  Chengdu|           2501.08|
|       3|    141|       Food|753.52|       5| Shenzhen|            3767.6|
|      11|    128|Electronics|431.19|       5| Shenzhen|           2155.95|
|      14|     58|      Books|819.95|       5|  Beijing|           4099.75|
|      16|     59|Electronics| 630.4|       4| Shenzhen|            2521.6|
|      18|    141|     Sports|701.34|       4| Shenzhen|           2805.36|
|      20|     39|     Sports|961.06|       5| Shenzhen| 4805.299999999999|
|      22|    131|       Food|809.88|       5|  Beijing|            4049.4|
|      25|    187|       Food|940.74|       3| Shenzhen|2822.2200000000003|
|      32|     58|       Food|592.06|       5|  Chengdu|2960.2999999999997|
|      34|    151|   Clothing|512.62|       4| Shanghai|           2050.48|
|      36|     19|Electronics|835.46|       3|  Beijing|           2506.38|
|      38|     84|Electronics|502.59|       4|  Chengdu|           2010.36|
|      49|     31|     Sports|798.64|       3|  Beijing|           2395.92|
|      54|    193|     Sports|733.66|       5|Guangzhou|3668.2999999999997|
|      63|    200|       Food|611.18|       5|  Beijing|3055.8999999999996|
|      64|    104|   Clothing|611.71|       5| Shenzhen|           3058.55|
|      65|    110|Electronics| 689.6|       5|Guangzhou|            3448.0|
|      66|    100|   Clothing|947.59|       3|  Beijing|           2842.77|
+--------+-------+-----------+------+--------+---------+-----------------+
only showing top 20 rows
Analysis 6: Top 10 users
```

Figure 12. Orders with an amount greater than 2,000 yuan.

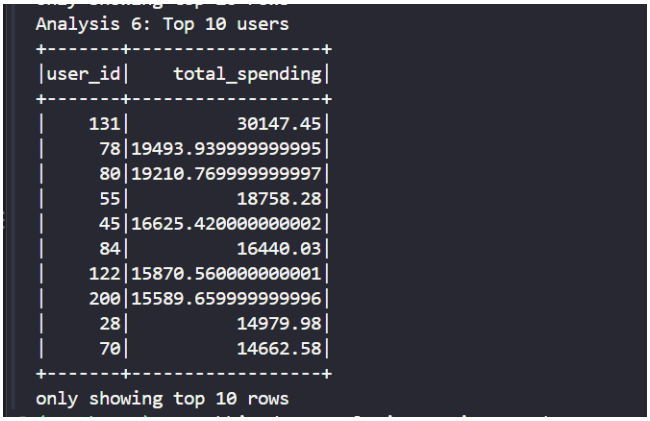
The output displayed the first 20 high-value orders (show() prints only 20 rows by default, so the total number of qualifying orders is not shown). Such a filter could be used in a real retail system to identify large transactions for further analysis. In this lab, the results are simulated and are not actual customer transactions.

4.9 Analysis 6 — Top 10 users by total spending

I grouped orders by user_id, summed the order amounts for each user, sorted the results in descending order, and displayed the first ten users:

```
df.groupBy("user_id") \
  .agg(sum("amount").alias("total_spending")) \
  .orderBy(col("total_spending").desc()) \
  .show(10)
```

This calculation adds together the amounts of all orders associated with each user. Sorting by total_spending in descending order puts the highest-spending users at the top.



```
Analysis 6: Top 10 users
+-------+------------------+
|user_id|    total_spending|
+-------+------------------+
|    131|          30147.45|
|     78|19493.939999999995|
|     80|19210.769999999997|
|     55|          18758.28|
|     45|16625.420000000002|
|     84|          16440.03|
|    122|15870.560000000001|
|    200|15589.659999999996|
|     28|          14979.98|
|     70|          14662.58|
+-------+------------------+
only showing top 10 rows
```

Figure 13. Ten users with the highest total spending.

The user with the highest total spending in my output was 131, with a total of 30,147.45 yuan. This type of analysis can help a business identify high-value customer groups, although further information would be required before making decisions about real customers.

4.10 Interpretation of findings

The assignment asks for the business implications of at least two findings. The following paragraphs should be completed with the actual values from my outputs.

Finding 1 — Revenue by category: The category with the highest total revenue was Clothing, with 363,458.85 yuan in revenue. This suggests that the category contributed the largest share of sales value in this generated dataset. A real business could investigate product availability, pricing, and demand in this category. Since the data is simulated, the result is an exercise in analysis rather than a real market conclusion.

Finding 2 — City or customer spending: The results showed that user 131 had the highest total spending, 30,147.45 yuan, about 55% more than the second-highest user (user 78, 19,493.94 yuan). This could help a business compare sales across locations or understand how order values are distributed among users. Before making a business decision, a company would need to validate the data and consider factors such as time period, marketing activity, and customer demographics, none of which are included in this dataset.

---

5. Task 3: Query the Data with Spark SQL

5.1 Registering the temporary view

After loading the dataset and creating the amount column, I registered the DataFrame as a temporary SQL view:

```
df.createOrReplaceTempView("orders")
```

This makes the DataFrame available to Spark SQL under the name orders for the lifetime of the Spark session. I could then use SQL statements to filter, group, aggregate, and sort the order records.

5.2 Query 1 — Orders with unit price above 500 yuan

```
SELECT *
FROM orders
WHERE price > 500
```

This query selects orders whose unit price is greater than 500 yuan. It does not filter by the calculated amount column. The WHERE clause keeps only records that meet the condition.

```
Query 1: Orders with unit price above 500
+--------+-------+-----------+------+--------+---------+------------------+
|order_id|user_id|   category| price|quantity|     city|            amount|
+--------+-------+-----------+------+--------+---------+------------------+
|       1|     31|   Clothing|507.49|       5|  Beijing|           2537.45|
|       2|     58|     Sports|625.27|       4|  Chengdu|           2501.08|
|       3|    141|       Food|753.52|       5| Shenzhen|            3767.6|
|       4|     62|Electronics|618.06|       1|Guangzhou|            618.06|
|       7|    191|     Sports|641.34|       3|  Beijing|           1924.02|
|      13|    123|     Sports|522.81|       1|  Chengdu|            522.81|
|      14|     58|      Books|819.95|       5|  Beijing|           4099.75|
|      16|     59|Electronics| 630.4|       4| Shenzhen|            2521.6|
|      18|    141|     Sports|701.34|       4| Shenzhen|           2805.36|
|      19|     52|   Clothing|833.52|       1|  Beijing|            833.52|
|      20|     39|     Sports|961.06|       5| Shenzhen| 4805.299999999999|
|      22|    131|       Food|809.88|       5|  Beijing|            4049.4|
|      25|    187|       Food|940.74|       3| Shenzhen|2822.2200000000003|
|      28|    140|Electronics|614.03|       2| Shanghai|           1228.06|
|      31|    148|       Food|692.86|       2|  Chengdu|           1385.72|
|      32|     58|       Food|592.06|       5|  Chengdu|2960.2999999999997|
|      34|    151|   Clothing|512.62|       4| Shanghai|           2050.48|
|      36|     19|Electronics|835.46|       3|  Beijing|           2506.38|
|      38|     84|Electronics|502.59|       4|  Chengdu|           2010.36|
|      41|    174|Electronics|551.88|       1| Shanghai|            551.88|
+--------+-------+-----------+------+--------+---------+------------------+
only showing top 20 rows
```

Figure 14. Orders with a unit price greater than 500 yuan.

The output displayed 20 rows (Spark's show() prints only the first 20 by default), all with a unit price above 500 yuan. This query demonstrates how SQL can filter rows using a condition.

5.3 Query 2 — Count orders by category

```
SELECT category, COUNT(*) AS order_count
FROM orders
GROUP BY category
ORDER BY order_count DESC
```

GROUP BY combines orders from the same category. COUNT(*) counts the rows in each group, and ORDER BY order_count DESC sorts categories from the largest count to the smallest.

```
Query 2: Order count by category
+-----------+-----------+
|   category|order_count|
+-----------+-----------+
|Electronics|        222|
|   Clothing|        213|
|       Food|        196|
|     Sports|        191|
|      Books|        178|
+-----------+-----------+
```

Figure 15. SQL order counts grouped by category.

The output provides the order count for each category and makes it easy to compare categories by order volume.

5.4 Query 3 — Total revenue by city

```
SELECT city, SUM(amount) AS total_revenue
FROM orders
GROUP BY city
ORDER BY total_revenue DESC
```

This query groups the orders by city and calculates the sum of amount in each group. The results are sorted by revenue in descending order.

```
Query 3: Revenue by city
+---------+------------------+
|     city|     total_revenue|
+---------+------------------+
| Shenzhen| 308339.0900000001|
|Guangzhou| 305989.2299999999|
| Shanghai| 302391.1199999998|
|  Beijing|290602.03999999986|
|  Chengdu| 283151.3300000002|
+---------+------------------+
```

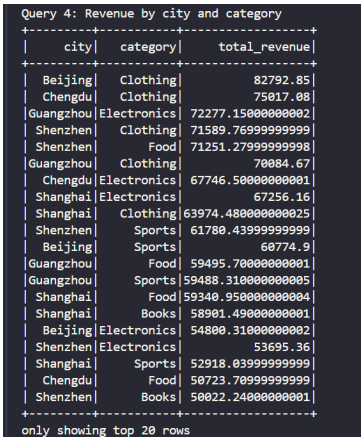
Figure 16. Total revenue by city using Spark SQL.

The city with the highest revenue in my output was Shenzhen, with 308,339.09 yuan. This query uses grouping, aggregation, and sorting in SQL.

5.5 Query 4 — Revenue by city and category

```
SELECT
  city,
  category,
  SUM(amount) AS total_revenue
FROM orders
GROUP BY city, category
ORDER BY total_revenue DESC
```

This query groups records by both city and category. Each result represents the total revenue for one city-category combination. Sorting the results in descending order places the combinations with the highest revenue first.



```
Query 4: Revenue by city and category
+---------+-----------+------------------+
|     city|   category|     total_revenue|
+---------+-----------+------------------+
|  Beijing|   Clothing|          82792.85|
|  Chengdu|   Clothing|          75017.08|
|Guangzhou|Electronics| 72277.15000000002|
| Shenzhen|   Clothing| 71589.76999999999|
| Shenzhen|       Food| 71251.27999999998|
|Guangzhou|   Clothing|          70084.67|
|  Chengdu|Electronics| 67746.50000000001|
| Shanghai|Electronics|          67256.16|
| Shanghai|   Clothing|63974.480000000025|
| Shenzhen|     Sports| 61780.43999999999|
|  Beijing|     Sports|           60774.9|
|Guangzhou|       Food| 59495.70000000001|
|Guangzhou|     Sports|59488.310000000005|
| Shanghai|       Food|59340.950000000004|
| Shanghai|      Books| 58901.49000000001|
|  Beijing|Electronics| 54800.31000000002|
| Shenzhen|Electronics|          53695.36|
| Shanghai|     Sports| 52918.03999999999|
|  Chengdu|       Food| 50723.70999999999|
| Shenzhen|      Books| 50022.24000000001|
+---------+-----------+------------------+
only showing top 20 rows
```

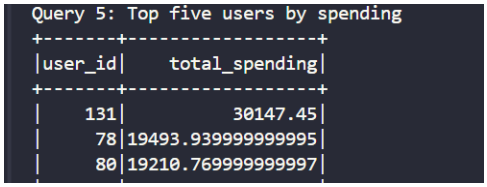
Figure 17. Revenue for each city and product-category combination.

This combined view can reveal differences in category performance between cities. Any interpretation should be based on the actual rows returned by the query.

5.6 Query 5 — Top five users by total spending

```sql
SELECT user_id, SUM(amount) AS total_spending
FROM orders
GROUP BY user_id
ORDER BY total_spending DESC
LIMIT 5
```

The query groups all orders by user, sums each user's order amounts, sorts by total spending, and uses `LIMIT 5` to return only the five highest totals.



```
Query 5: Top five users by spending
+-------+------------------+
|user_id|    total_spending|
+-------+------------------+
|    131|          30147.45|
|     78|19493.939999999995|
|     80|19210.769999999997|
```

Figure 18. Five users with the highest total spending using Spark SQL.

This query provides a compact ranking of the five users with the highest spending totals in the generated dataset.

5.7 Comparison of the DataFrame API and Spark SQL

I implemented the city-revenue calculation using both the DataFrame API and Spark SQL.

DataFrame API:

```python
dataframe_result = (
    df.groupBy("city")
    .agg(sum("amount").alias("total_revenue"))
    .orderBy(col("total_revenue").desc())
)
```

Spark SQL:

```sql
SELECT city, SUM(amount) AS total_revenue
FROM orders
GROUP BY city
ORDER BY total_revenue DESC
```

Both approaches group the data by city, calculate total revenue, and sort the result from highest to lowest. The DataFrame API expresses the calculation through Python method calls and column expressions. Spark SQL expresses the same calculation using SQL syntax, which can be familiar to people who work with relational databases.

```
DataFrame API: Revenue by city
+---------+-----------------+
|     city|    total_revenue|
+---------+-----------------+
| Shenzhen| 308339.0900000001|
|Guangzhou| 305989.2299999999|
| Shanghai| 302391.1199999998|
|  Beijing|290602.03999999986|
|  Chengdu| 283151.3300000002|
+---------+-----------------+


Spark SQL: Revenue by city
+---------+-----------------+
|     city|    total_revenue|
+---------+-----------------+
| Shenzhen| 308339.0900000001|
|Guangzhou| 305989.2299999999|
| Shanghai| 302391.1199999998|
|  Beijing|290602.03999999986|
|  Chengdu| 283151.3300000002|
```

Figure 19. Comparison of the DataFrame API and Spark SQL results.

The two results should contain the same city names and revenue values because they perform the same calculation on the same DataFrame. In my run, the comparison showed that the DataFrame API and Spark SQL returned identical city names, revenue values, and ordering (Shenzhen, Guangzhou, Shanghai, Beijing, Chengdu). The DataFrame API can be convenient when transformations are constructed programmatically in Python. SQL can be convenient when a query is naturally expressed as a relational statement. Both approaches are supported by Spark's structured data engine.

5.8 Physical execution plan

I used result3.explain() to display the physical execution plan for the city-revenue query. The output included operations such as FileScan, Project, HashAggregate, Exchange, Sort, and AdaptiveSparkPlan.

```
Execution plan for revenue by city
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [total_revenue#65 DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(total_revenue#65 DESC NULLS LAST, 200), ENSURE_REQUIREMENTS, [plan_id=396]
      +- HashAggregate(keys=[city#22], functions=[sum(amount#24)])
         +- Exchange hashpartitioning(city#22, 200), ENSURE_REQUIREMENTS, [plan_id=393]
            +- HashAggregate(keys=[city#22], functions=[partial_sum(amount#24)])
               +- Project [city#22, (price#20 * cast(quantity#21 as double)) AS amount#24]
```

Figure 20. Physical execution plan for the revenue-by-city query.

The main operations in the observed plan have the following meanings:

- FileScan csv: reads the required columns from the CSV source.
- Project: calculates the amount expression from price multiplied by quantity.
- HashAggregate: performs aggregation, including partial sums and final sums.
- Exchange hashpartitioning: redistributes data by city so that records for each grouping key can be aggregated.
- Exchange rangepartitioning: redistributes the aggregated rows to support the required ordering.
- Sort: sorts the results by total_revenue in descending order.
- AdaptiveSparkPlan: indicates that Spark's adaptive query execution framework is involved in planning or executing the query.

The plan showed partial aggregation before the final aggregation. This allows Spark to calculate partial sums before combining them into the final city totals. The exchanges indicate stages where data is redistributed between partitions. The plan therefore helps explain the operations Spark uses to produce the requested result.

The plan also displayed 200 for the relevant exchange partitioning settings. This is the number of shuffle partitions specified in those exchanges in the displayed plan; it is not the number of cities or orders.

6. Task 4: Reflection Questions and Lab Summary

6.1 Question 1 — How does local mode differ from cluster mode? Which mode is used in this lab?

In local mode, Spark runs on one computer and uses the computer's local resources. In cluster mode, Spark runs across a cluster of machines, with work distributed across multiple worker nodes under a cluster deployment configuration. Cluster mode is useful for larger workloads and distributed resources, while local mode is useful for development, testing, and learning.

This lab uses local mode because the Spark session is configured with .master("local[*]"). The * tells Spark to use the available logical CPU cores for local parallel processing. A separate Spark cluster is not required for this assignment.

6.2 Question 2 — How are RDDs, DataFrames, and Spark SQL related?

RDDs, DataFrames, and Spark SQL are related ways to work with data in Spark. An RDD (Resilient Distributed Dataset) is a lower-level distributed collection that supports transformations and actions. A DataFrame represents structured data organized into named columns, allowing operations such as filtering, grouping, aggregation, and sorting. Spark SQL provides a SQL interface for querying structured data.

In this lab, I used DataFrames to read and analyze the CSV file. I then registered the DataFrame as a temporary view and used Spark SQL to run queries against it. Both approaches operate within Spark and can use Spark's optimization and execution engine. DataFrames and Spark SQL are especially convenient for structured data because the column names and data types are explicit.

6.3 Question 3 — What is the difference between a transformation and an action? Give two examples of each from this lab.

A transformation defines a new DataFrame or a new stage of processing from existing data. Spark generally evaluates transformations lazily, meaning it can build a plan before executing the work required to produce a result. An action requests a result or output and triggers the required computation.

Two transformations from this lab are:

1. withColumn("amount", col("price") * col("quantity")), which defines a calculated column.
2. filter(col("amount") > 2000), which defines a DataFrame containing only high-value orders.

Two actions from this lab are:

1. df.show(), which displays records.
2. df.count(), which calculates the number of rows.

The SQL queries also produce results when actions such as show() are called on their resulting DataFrames. Understanding lazy evaluation helps explain why defining a transformation does not necessarily mean that Spark has already processed every record.

6.4 Question 4 — Can a dataset of only 1,000 orders demonstrate that Spark is faster than regular Python processing? Why or why not?

A dataset containing only 1,000 orders is not enough to demonstrate that Spark is faster than regular Python processing. Spark has startup, planning, and execution overhead. For a small dataset, ordinary Python may complete a simple operation more quickly because it does not need to initialize Spark's execution environment.

Spark is designed to support scalable data processing and can distribute work across available resources. To compare performance fairly, I would need to implement equivalent calculations in regular Python and Spark, use the same dataset and output requirements, measure repeated runs, account for startup overhead, and test several dataset sizes. The results could then show how performance changes as the workload grows. This lab demonstrates Spark's APIs and execution concepts, not proof that Spark is always faster for small datasets.

6.5 Question 5 — Describe a troubleshooting experience or explain how correctness was verified.

During this lab, I encountered a local Spark startup error containing the message Invalid Spark URL, associated with the machine hostname. I addressed the local network-binding issue by setting the following environment variables in PowerShell before rerunning the Spark script:

```
$env:SPARK_LOCAL_HOSTNAME = "localhost"
$env:SPARK_LOCAL_IP = "127.0.0.1"
```

I also encountered a CSV path issue because the generator writes orders.csv relative to the current working directory. I addressed this by generating the CSV in the project's data directory and using a path based on the script location in the analysis code. This made the dataset path explicit instead of depending on the terminal's current directory.

To verify correctness, I checked that the generated dataset contained 1,000 records, inspected the column schema and null counts, and reviewed the calculated amount values. I also compared the city-revenue output from the DataFrame API with the corresponding Spark SQL query. The two approaches should return matching city revenue totals because they perform the same aggregation on the same input data. I used the physical execution plan to inspect how Spark planned the SQL query.

6.6 Personal reflection (at least 300 words)

This lab helped me understand how Apache Spark can be used to process structured data through PySpark. Before completing the tasks, I needed to become familiar with the role of SparkSession, how Spark reads a CSV file, and how a DataFrame represents data in named columns. The first experiment was useful because it provided a small and clear way to verify that Spark could start, create a DataFrame, display records, and count them.

The order-analysis task helped me practice several common data-processing operations. I generated a dataset of 1,000 simulated orders and inspected its schema and record count before performing calculations. Adding the amount column showed how a derived value can be calculated with column expressions. Grouping by category or city made it possible to summarize order counts and revenue, while filtering and sorting helped identify high-value orders and users with the highest spending. Checking for missing values was also important because incomplete data can affect calculations and interpretation.

The Spark SQL task showed that structured data can be queried using SQL as well as the DataFrame API. Although the syntax differs, both approaches can express the same logical analysis. Comparing the revenue-by-city results helped me check that the two implementations produced consistent outputs. The physical execution plan gave me a better understanding of how Spark carries out the query. Operations such as FileScan, Project, HashAggregate, Exchange, and Sort represent stages of reading, calculating, aggregating, redistributing, and ordering data. This also helped me understand that Spark does more than execute each Python line independently; it builds and executes a plan for the requested computation.

One practical challenge was ensuring that Spark could initialize correctly on my Windows computer. The hostname-related error required local network-binding settings. I also had to pay attention to the location of orders.csv, because relative file paths depend on the working directory. These issues taught me to distinguish between a problem in the data-analysis logic and a problem in the environment or file paths.

Finally, this lab helped me understand the difference between transformations and actions. Transformations describe operations to be performed, while actions such as show() and count() request results and trigger computation. The dataset was small, so I cannot conclude that Spark is faster than regular Python based on this experiment. Instead, the main outcome was learning Spark's basic workflow and how to use DataFrames, SQL, and execution plans to analyze structured data.

---

7. Conclusion

In this lab, I practiced setting up and verifying a PySpark environment, creating a Spark session, and processing a simulated online-retail dataset containing 1,000 orders. I used the DataFrame API to inspect the data, check for missing values, calculate order amounts, summarize order counts and revenue, filter high-value orders, and rank users by spending. I then used Spark SQL to perform five equivalent types of structured queries.

The comparison between the DataFrame API and Spark SQL demonstrated that the same analysis can be expressed in different styles. Examining the physical execution plan helped me identify how Spark reads the data, calculates derived values, aggregates groups, redistributes data, and sorts the results. The lab also gave me practical experience with local Spark configuration and file-path troubleshooting.

Overall, the lab strengthened my understanding of PySpark's basic workflow, structured data analysis, and the relationship between transformations, actions, and execution plans.

---

8. References and AI-Use Acknowledgement

8.1 References

1. Apache Software Foundation. *Apache Spark Documentation*. https://spark.apache.org/docs/latest/
2. Apache Software Foundation. *PySpark Installation*. https://spark.apache.org/docs/latest/api/python/getting_started/install.html
3. Apache Software Foundation. *Spark Quick Start*. https://spark.apache.org/docs/latest/quick-start.html
4. Apache Software Foundation. *DataFrame Quick Start*. https://spark.apache.org/docs/latest/api/python/getting_started/quickstart_df.html
5. Apache Software Foundation. *Spark SQL Programming Guide*. https://spark.apache.org/docs/latest/sql-programming-guide.html
6. Apache Software Foundation. *RDD Programming Guide*. https://spark.apache.org/docs/latest/rdd-programming-guide.html
7. Course handout: *Laboratory Assignment 1: Setting Up Apache Spark and Performing Data Analysis*, Big Data Analytics and Applications.

8.2 AI-use acknowledgement

I used an AI assistant as a learning aid to clarify PySpark concepts, understand error messages, organize the report, and improve the explanations. I reviewed the code and explanations, ran the programs in my own environment, and used my actual outputs for the results and screenshots. I am responsible for checking the accuracy of the submitted work and for being able to explain the code and analysis.
